



Instituto Tecnológico De Culiacán

Tópicos De Inteligencia Artificial

Sistema de enrutamiento para tienda de conveniencia

Integrantes:

Robles Zamora Anette Leticia

Ruelas Gutiérrez Maia Paulina

Maestro:

Mora Félix Zuriel Dathan

12/Octubre/2025

1. Análisis del Problema Logístico

La cadena de tiendas en estudio cuenta con 90 puntos de venta y 10 centros de distribución en la ciudad de Culiacán, Sinaloa. Sin embargo, para efectos del modelo computacional se consideró un único centro de distribución como punto de partida y llegada (depot) y 90 tiendas como puntos de entrega.

El objetivo principal del proyecto es optimizar las rutas de distribución de productos comestibles desde el centro de distribución hacia las tiendas, minimizando el costo total y la distancia recorrida por los vehículos de reparto.

Actualmente, la red logística enfrenta los siguientes desafíos principales:

- Rutas no optimizadas que incrementan el costo operativo.
- Alta congestión vehicular en horas pico que afecta los tiempos de entrega.
- Necesidad de reducir los kilómetros recorridos para disminuir gastos de combustible y mantenimiento.

El problema se enmarca en el ámbito de la distribución urbana, donde los factores clave incluyen los patrones de tráfico, la eficiencia de las rutas y la minimización de costos de transporte, considerando que los productos son perecederos y deben entregarse con rapidez.

2. Modelado del Problema como Caso de Optimización Combinatoria

El problema se modela como una variación del Problema del Viajante (TSP, Travelling Salesman Problem), donde un solo vehículo debe visitar todas las tiendas una vez y regresar al centro de distribución minimizando el costo total del recorrido.

Parámetros del modelo:

- $N = 90$: número de tiendas.
- D_0 : centro de distribución (nodo inicial y final).
- c_{ij} : costo de recorrer la distancia entre los puntos i y j .
- $COSTO_POR_KM = 8.2$ pesos/km (calculado a partir de precios locales de combustible y mantenimiento).

Restricciones:

- No existen restricciones adicionales: todos los puntos deben ser visitados exactamente una vez.

- El recorrido inicia y termina en el mismo centro de distribución.

El modelo es un problema combinatorio NP-difícil, por lo que no es factible resolverlo de forma exacta con programación entera para $N = 90$ puntos. Por ello, se aplican técnicas heurísticas.

3. Selección y Justificación del Algoritmo Heurístico

Tras analizar diferentes métodos de búsqueda, se seleccionó el algoritmo de Recocido Simulado (Simulated Annealing).

Comparación de heurísticas:

Criterio	Recocido Simulado	Búsqueda tabú
Adaptabilidad	Alta, permite escapar de óptimos locales	Moderada
Facilidad de implementación	Simple y flexible	Requiere estructuras adicionales (lista tabú)
Ajuste de parámetros	Basado en temperatura y tasa de enfriamiento	Basado en tamaño de lista tabú
Eficiencia en TSP grandes	Alta	Media

El recocido simulado fue elegido porque permite explorar el espacio de soluciones de manera controlada, aceptando ocasionalmente soluciones peores para escapar de óptimos locales. Esto es ideal para el problema de rutas en Culiacán, donde la cantidad de combinaciones posibles es extremadamente grande ($90!$).

4. Desarrollo e Implementación Computacional

El proyecto fue desarrollado en Python, empleando librerías estándar y la librería Folium para la visualización geográfica de las rutas.

Generación de coordenadas

El primer programa convierte un bloque de texto con coordenadas de tiendas y centros de distribución en un archivo CSV:

```

centros, tiendas = separar_lineas(DATOS)

if not centros:
    print("No encontré centros de distribución. Revisa tu bloque de texto.", file=sys.stderr)
    sys.exit(1)

# Tomamos "Centro de Distribución 1" como depot si existe, si no el primero
depot = next((c for c in centros if c["nombre"].endswith("1")), centros[0])

```

El archivo resultante contiene los campos:
id, nombre, lat, lon, donde id = 0 representa el centro de distribución.

El módulo **Utilidades.py** implementa la fórmula de Haversine para calcular la distancia entre coordenadas geográficas, y define el costo por kilómetro:

```

def haversine_km(lat1, lon1, lat2, lon2):
    # fórmula Haversine
    # distancia aproximada sobre la Tierra (radio ~6371 km)
    R = 6371.0
    dlat = math.radians(lat2 - lat1)
    dlon = math.radians(lon2 - lon1)
    a = (math.sin(dlat/2)**2 +
         math.cos(math.radians(lat1)) * math.cos(math.radians(lat2)) * math.sin(dlon/2)**2)
    c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
    return R * c

```

Algoritmo de Recocido Simulado

El módulo **Recocido.py** contiene la lógica principal del algoritmo:

```

def recocido(puntos, T_inicial=100.0, T_min=0.01, alfa=0.995, max_iter=20000, imprimir_cada=1000):
    n = len(puntos)
    ruta_actual = hacer_ruta_inicial(n)
    costo_actual = costo_ruta(puntos, ruta_actual)

    mejor_ruta = ruta_actual[:]
    mejor_costo = costo_actual

    T = T_inicial
    iteracion = 0

    print(f"Inicio | T={T:.2f} | costo=${costo_actual:.2f} | ruta={ruta_actual}")

    while T > T_min and iteracion < max_iter:
        iteracion += 1

        # generamos un vecino
        vecino = vecino_aleatorio(ruta_actual)
        costo_vecino = costo_ruta(puntos, vecino)

        # calculamos delta
        delta = costo_vecino - costo_actual

        # criterio de aceptación
        if delta <= 0:
            # mejora: aceptamos siempre
            ruta_actual, costo_actual = vecino, costo_vecino

```

```

else:
    # empeora: aceptamos a veces con probabilidad exp(-delta/T)
    if random.random() < math.exp(-delta / T):
        ruta_actual, costo_actual = vecino, costo_vecino

# actualizamos mejor global
if costo_actual < mejor_costo:
    mejor_ruta, mejor_costo = ruta_actual[:], costo_actual

# enfriamos la temperatura
T *= alfa

```

El algoritmo inicia con una ruta secuencial ($0 \rightarrow 1 \rightarrow 2 \dots \rightarrow 0$), y en cada iteración aplica movimientos “swap” y 2-opt para generar soluciones vecinas.

La temperatura T decrece gradualmente con un factor $\alpha = 0.995$, lo que permite pasar de exploración a explotación.

Ejecución principal y visualización

El archivo principal (run.py) ejecuta el recocido y genera un mapa HTML interactivo con la mejor ruta encontrada usando Folium:

```

# Ejecuta recocido
mejor_ruta, mejor_costo, iters = recocido(
    puntos,
    T_inicial=100.0,
    T_min=0.01,
    alfa=0.995,
    max_iter=10000,
    imprimir_cada=1000 # imprime progreso cada N iteraciones
)

km_totales = mejor_costo / COSTO_POR_KM

```

```

# Guarda el mapa como archivo HTML
mapa.save("ruta.html")
print("\nMapa generado: ruta.html")

```

5. Evaluación del Rendimiento

Métrica	Descripción
Mejor ruta	Orden de visita de los puntos optimizada
Km totales aproximados	Distancia total recorrida en kilómetros
Costo total aproximado	Costo en pesos mexicanos
Iteraciones usadas	Número de iteraciones ejecutadas hasta convergencia

El algoritmo logra reducir significativamente los kilómetros recorridos comparado con rutas aleatorias o secuenciales, optimizando tanto el costo total como la eficiencia operativa.

El mapa generado (ruta.html) permite visualizar la secuencia óptima de entrega en un entorno geográfico real de Culiacán.

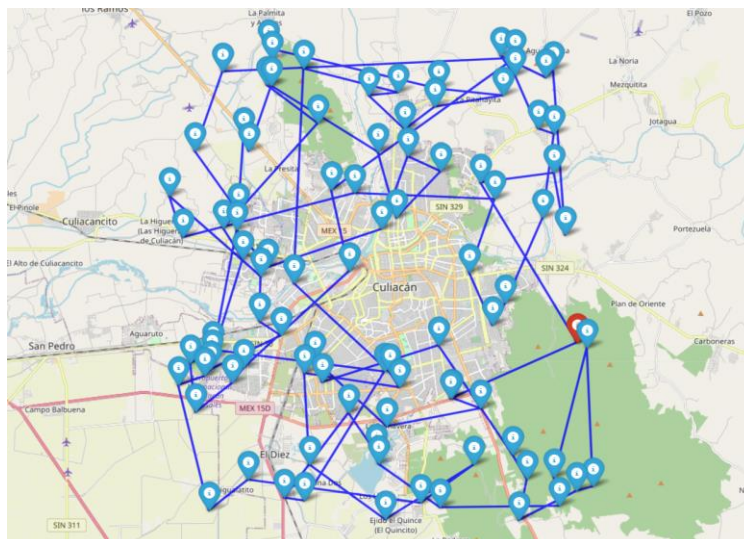
Pruebas de código:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\anett\OneDrive\Desktop\proyecto> & C:\Users\anett\AppData\Local\Programs\Python\Python311\python.exe c:/User
s/anett/OneDrive/Desktop/proyecto/run.py
Inicio | T=100.00 | costo=$8644.98 | ruta=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21,
22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51,
52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81,
82, 83, 84, 85, 86, 87, 88, 89, 90, 0]
Iter   1 | T= 99.500 | costo=$ 8625.71 | mejor=$ 8625.71
Iter  100 | T=  0.665 | costo=$ 3567.24 | mejor=$ 3567.24

Mapa generado: ruta.html

===== RESULTADO =====
Mejor ruta (orden de IDs): [0, 83, 13, 70, 4, 88, 2, 71, 24, 73, 27, 7, 25, 60, 86, 12, 41, 11, 67, 53, 65, 20, 42, 30,
32, 45, 46, 74, 39, 79, 8, 80, 29, 78, 49, 26, 43, 38, 72, 34, 37, 64, 85, 6, 81, 15, 1, 22, 66, 61, 68, 87, 90, 31, 84,
35, 50, 58, 62, 14, 23, 33, 57, 55, 36, 75, 19, 40, 63, 9, 52, 21, 56, 77, 76, 5, 10, 69, 59, 44, 3, 48, 51, 54, 47, 16
, 82, 17, 18, 89, 28, 0]
Km totales aprox: 342.44 km
Costo total aprox: $2807.97
Iteraciones usadas: 1838
PS C:\Users\anett\OneDrive\Desktop\proyecto> █
```



Conclusiones:

El recocido simulado es una técnica eficaz para resolver problemas logísticos de gran escala como la distribución de productos comestibles.

Los resultados obtenidos muestran que es posible minimizar costos y distancias sin necesidad de métodos exactos computacionalmente costosos.

Los resultados de este proyecto muestran que, al ajustar bien los parámetros del algoritmo y representar correctamente el problema, se pueden reducir los costos operativos, las distancias recorridas y los tiempos de entrega. Además, el recocido simulado es muy flexible y puede adaptarse fácilmente a distintas condiciones del sistema, como la capacidad de los camiones, los horarios de entrega, etc.